Spec: `rails_airtable_sync`

1. Purpose

`rails_airtable_sync` is a Ruby on Rails gem that synchronizes application data from Rails models to Airtable records in a **one-way direction only**:

Rails → Airtable

The gem is intended for teams that use Airtable as a reporting, operations, or lightweight business-facing interface, while Rails remains the source of truth.

2. Goals

The gem must:

- Push Rails model data to Airtable reliably.
- Preserve data integrity across sync runs.
- Enforce predictable type conversion between Rails and Airtable fields.
- Provide robust retry and failure handling.
- Support idempotent operations.
- Prevent accidental destructive updates.
- Support auto-creating and modifying Airtable schemas by default.
- Offer observability for sync status, failures, schema changes, and drift.

3. Non-Goals

The gem will not:

- Perform two-way sync.
- Treat Airtable as a source of truth.
- perform uncontrolled or unsafe schema mutations without validation, guardrails, or auditability.
- resolve business conflicts from external Airtable edits beyond configured policy.
- support arbitrary bidirectional merging.

---

4. Core Principles

4.1 Rails is the source of truth

All synced records originate from Rails. Airtable data may be overwritten by Rails on subsequent syncs according to configured sync rules.

4.2 Idempotent syncs

Running the same sync repeatedly with unchanged source data must produce the same Airtable state and must not create duplicate records.

4.3 Explicit mappings

All model-to-table and field-to-field mappings must be explicitly defined. No implicit “sync all attributes” behavior by default.

4.4 Safe by default

The gem must default to conservative behavior for destructive actions:

- no record deletes unless explicitly enabled,
- schema creation and compatible schema updates enabled by default,
- no destructive schema changes unless explicitly enabled,
- no silent type coercion failures,
- no swallowing of API errors.

4.5 Auditable operations

All sync attempts must be traceable with structured logs and optional persisted sync records.

4.6 Schema management is supported by default

The gem should support **auto-creating and modifying Airtable schemas by default** to reduce setup friction and keep Rails mappings aligned with Airtable structure.

Because schema mutation is operationally sensitive, it must still be:

- explicit in logs,
- validated before execution,
- idempotent,
- reversible where possible,
- protected by guardrails for destructive changes.

---

5. High-Level Architecture

5.1 Main components

A. Configuration Layer

Defines:

- Airtable API credentials
- base and table mappings
- model scopes
- field mappings
- error handling policy
- retry policy
- deletion policy
- sync mode
- schema management policy
- field creation/update rules
- destructive schema change policy
- schema drift reconciliation behavior

B. Sync Engine

Responsible for:

- extracting source records from Rails
- transforming data into Airtable-compatible payloads
- validating mapped values
- reconciling schema before data sync
- creating/updating Airtable records
- tracking sync state and failures

C. Type System / Serializer Layer

Responsible for:

- converting Rails values into Airtable field-compatible representations
- validating compatibility before API submission
- producing deterministic serialization output

D. State Tracking Layer

Responsible for:

- mapping Rails records to Airtable record IDs
- storing sync checksum/version
- tracking last sync timestamps and failure states

E. Error Handling and Retry Layer

Responsible for:

- classifying failures
- retrying transient failures
- quarantining permanent failures
- surfacing errors clearly

F. Instrumentation Layer

Responsible for:

- logs
- notifications
- metrics
- optional ActiveSupport::Notifications hooks

G. Schema Reconciliation Layer

Responsible for:

- inspecting Airtable base/table metadata,
- creating missing tables if allowed,
- creating missing fields if allowed,
- updating compatible field definitions,
- detecting unsafe or destructive schema drift,
- blocking sync when remote schema cannot safely support the mapping.

---

6. Public API

6.1 Gem configuration

```
RailsAirtableSync.configure do |config|
 config.api_key = ENV["AIRTABLE_API_KEY"]
 config.base_id = ENV["AIRTABLE_BASE_ID"]

 config.timeout = 10
 config.open_timeout = 3

 config.max_retries = 3
 config.retry_backoff = :exponential
 config.retry_jitter = true

 config.fail_fast = false
 config.enable_deletes = false
 config.delete_strategy = :archive_flag
```

```
 config.strict_types = true
 config.verify_after_write = true
 config.validate_remote_schema = true

 config.persist_sync_state = true
 config.use_advisory_locks = true

 config.auto_manage_schema = true
 config.auto_create_tables = true
 config.auto_create_fields = true
 config.auto_update_fields = true
 config.allow_destructive_schema_changes = false
 config.schema_conflict_policy = :fail

 config.on_validation_error = :mark_failed
 config.on_consistency_error = :quarantine

 config.logger = Rails.logger
end
```

6.2 Model mapping DSL

```
class Customer < ApplicationRecord
 include RailsAirtableSync::Model

 airtable_sync table: "Customers" do
   scope { where(active: true) }
   record_key :id
   external_id_field "Rails ID"

   field "Rails ID", from: :id, type: :integer, nullable: false
   field "Email", from: :email, type: :email, nullable: false
   field "Full Name", from: :full_name, type: :string
   field "Subscribed", from: :subscribed, type: :boolean
   field "Signup Date", from: :created_at, type: :datetime
   field "Tags", from: :tag_list, type: :multi_select,
         allowed_values: %w[vip trial churn_risk]

   checksum_fields :email, :full_name, :subscribed, :created_at, :tag_list
 end
end
```

6.3 Sync execution API

```
RailsAirtableSync.sync(Customer)
RailsAirtableSync.sync(Customer, record: customer)
```

```
RailsAirtableSync.sync_all
RailsAirtableSync.enqueue_sync(Customer, record: customer)
```

6.4 Rake tasks

```
bin/rails airtable:sync[Customer]
bin/rails airtable:sync_all
bin/rails airtable:resync_failed
bin/rails airtable:verify[Customer]
```

---

7. Sync Modes

7.1 Full sync

Scans all records in configured scope and ensures Airtable matches Rails.

Use case:

- initial import
- repair jobs
- periodic reconciliation

7.2 Incremental sync

Sync only records changed since last successful sync or via callbacks/jobs.

Use case:

- day-to-day operation

7.3 Single-record sync

Sync an individual model instance.

Use case:

- after_commit job
- admin repair action

7.4 Verify-only mode

Compares Rails payload to Airtable without writing changes.

Use case:

- dry runs
- monitoring drift
- rollout validation

---

8. Record Identity and Integrity

8.1 External identity model

Each Rails record must map to exactly one Airtable record using a stable key.

Supported strategies:

Preferred: external ID field

Airtable contains a dedicated field such as `"Rails ID"` or `"External UUID"`.

Required persisted mapping

Gem stores a local mapping between:

- Rails model class
- Rails primary key
- Airtable record ID
- payload checksum
- last_synced_at

8.2 Duplicate prevention

The gem must prevent duplicate Airtable records by:

- requiring a stable external identifier,
- looking up existing Airtable records by external ID before create,
- maintaining local sync state,
- optionally verifying uniqueness in Airtable at sync time.

8.3 Integrity checks

Before writing:

- verify model record exists and is persisted,
- verify mapped fields are valid,
- verify primary identity field is present,
- verify serialized values comply with Airtable expectations.

After writing:

- verify returned Airtable record ID exists,
- verify returned fields match submitted payload where possible,
- persist sync state only after successful remote acknowledgement.

8.4 Schema integrity precondition

Before record sync, the gem must ensure the Airtable schema is compatible with the configured mapping.

If `auto_manage_schema` is enabled, the gem should first attempt to reconcile the schema.

If reconciliation fails or would require unsafe destructive mutation, the sync must not proceed for affected mappings.

---

9. Data Model for Local Sync State

Recommended internal table:

`airtable_sync_records`

Columns:

- `id`
- `syncable_type`
- `syncable_id`
- `airtable_table`
- `airtable_record_id`
- `external_key`
- `payload_checksum`
- `last_synced_at`
- `last_attempted_at`
- `last_error_class`
- `last_error_message`
- `failure_count`
- `status` (`pending`, `synced`, `failed`, `skipped`, `quarantined`)
- `created_at`
- `updated_at`

Indexes:

- unique index on `[:syncable_type, :syncable_id, :airtable_table]`
- unique index on `[:airtable_table, :external_key]`
- index on `status`
- index on `last_synced_at`

This table is the backbone for idempotency, observability, and retries.

---

10. Data Type Handling

10.1 Supported canonical types

The gem should define an internal type system:

- `:string`
- `:text`
- `:integer`
- `:float`
- `:decimal`
- `:boolean`
- `:date`
- `:datetime`
- `:email`
- `:url`
- `:phone`
- `:single_select`
- `:multi_select`
- `:json`
- `:attachment_url`
- `:lookup_string`
- `:formula_safe_string`

10.2 Type conversion rules

String/Text

- Convert to UTF-8 string.
- Strip invalid byte sequences.
- Enforce optional max length.
- `nil` remains `nil`.

Integer

- Accept Integer values only, or strings if strict coercion is disabled.
- Reject floats unless explicitly configured.
- Overflow must raise validation error.

Float/Decimal

- Preserve precision where possible.
- Decimal may be stringified to avoid Ruby float precision loss if required by Airtable API format.

Boolean

- Accept `true`/`false`.
- Reject ambiguous strings like `"yes"` unless custom coercion is enabled.

Date

- Serialize as ISO 8601 date: `YYYY-MM-DD`
- Time zone normalization required before truncation.

Datetime

- Serialize as ISO 8601 UTC timestamp.
- All times normalized to UTC before transmission.
- Nil allowed if field is optional.

Email/URL/Phone

- Validate format minimally before sync.
- Invalid values produce field-level validation errors.

Single Select

- Must match allowed configured enum values.
- Unknown values cause validation failure unless `on_unknown_select: :coerce_to_string` is enabled.

Multi Select

- Must be an array of strings.
- Values normalized and deduplicated.
- Unknown options handled per policy.

JSON

- Serialized to compact JSON string unless table explicitly expects long text.

Attachment URL

- Must be valid absolute URL.
- Only remote URLs supported unless custom upload extension exists.

10.3 Strict vs permissive typing

Default mode: **strict**

In strict mode:

- incompatible values fail before API call.

Permissive mode:

- configured coercions may be applied.

Example:

field "Revenue", from: :revenue_cents, type: :decimal, coerce: true

10.4 Null handling

Each field mapping must support:

- `nullable: true/false`
- optional default fallback
- optional omission on nil

Examples:

- `nil` may clear an Airtable field
- or `nil` may skip update for that field
- behavior must be explicit

10.5 Airtable schema type mapping

The gem must maintain a canonical mapping between gem field types and Airtable field definitions.

Example mappings:

- `:string`, `:text` → single line text / long text
- `:integer`, `:float`, `:decimal` → number
- `:boolean` → checkbox

- `:date` → date
- `:datetime` → date/time
- `:email` → email
- `:url` → URL
- `:phone` → phone number
- `:single_select` → single select
- `:multi_select` → multiple select
- `:json` → long text
- `:attachment_url` → attachment-compatible staging field or URL text fallback

The mapping layer must distinguish between:

- value serialization for record sync,
- field definition generation for schema sync.

---

11. Schema Validation and Auto-Reconciliation

11.1 Default behavior

By default, the gem should attempt to **auto-create or modify Airtable schemas** to match configured Rails mappings.

Default settings:

- `auto_manage_schema = true`
- `auto_create_tables = true`
- `auto_create_fields = true`
- `auto_update_fields = true`
- `allow_destructive_schema_changes = false`

11.2 Startup validation

On startup or before sync execution, the gem should:

- verify Airtable base accessibility,
- inspect the target table,
- create the table if missing and allowed,
- inspect remote fields,
- create missing fields,
- update compatible field settings where possible,

- detect incompatible or destructive schema drift.

11.3 Safe schema updates

Allowed by default:

- create missing tables,
- create missing fields,
- update non-destructive field metadata,
- extend select options,
- align field formatting/options where Airtable supports it.

Blocked by default:

- deleting fields,
- renaming fields with ambiguity,
- changing a field to an incompatible type,
- narrowing select options in a way that invalidates existing data,
- destructive replacement of populated fields.

11.4 Drift detection

The gem should detect:

- missing tables,
- missing fields,
- mismatched field types,
- missing select options,
- renamed or replaced fields,
- remote field definitions incompatible with the configured mapping.

11.5 Reconciliation outcomes

For each mapping element, reconciliation should produce one of:

- `:ok`
- `:created`
- `:updated`
- `:blocked`
- `:failed`

If a required field is `:blocked` or `:failed`, record sync for that table must stop.

11.6 Destructive schema policy

Destructive schema changes must require explicit opt-in:

config.allow_destructive_schema_changes = true

Supported destructive policies:

- `:fail` — block and raise
- `:ignore` — skip change and continue where safe
- `:replace` — replace incompatible field if explicitly permitted
- `:archive_and_replace` — create replacement field and mark old field deprecated

Recommended default:

- `schema_conflict_policy = :fail`

---

12. Sync Workflow

12.1 For each sync run

1. Load model sync configuration.
2. Inspect Airtable schema.
3. Reconcile schema if `auto_manage_schema` is enabled.
4. Fail the sync if required schema elements cannot be safely established.
5. Load Rails records from configured scope.
6. Build deterministic payload for each record.
7. Validate payload and field types.
8. Compute checksum from serialized payload.
9. Compare to last synced checksum.
10. If unchanged, skip write.
11. Resolve Airtable record identity.
12. Create or update Airtable record.
13. Validate API response.
14. Persist sync state.
15. Emit instrumentation events.

12.2 Checksum semantics

Checksum should be computed from:

- mapped field names
- serialized values
- stable sort order

Use SHA-256 of canonical JSON.

This avoids unnecessary API writes and supports idempotency.

---

13. Create, Update, and Delete Semantics

13.1 Create

If no existing Airtable record is found:

- create record,
- store Airtable record ID,
- store checksum.

13.2 Update

If record exists and checksum differs:

- update only changed fields by default,
- optionally support full overwrite mode.

13.3 Delete

Disabled by default.

If enabled, deletion policy options:

- `:none`
- `:clear_fields`
- `:archive_flag`
- `:delete_record`

Recommended default when enabled: `:archive_flag`

Hard deletion should require explicit config:

```
config.enable_deletes = true
config.delete_strategy = :delete_record
```

13.4 External edits in Airtable

Since Rails is source of truth:

- external Airtable edits are overwritten on next sync for mapped fields.
- unmapped fields are preserved unless full overwrite mode is enabled.

Optional feature:

- detect drift between last synced checksum and current Airtable state for mapped fields and log warnings.

13.5 Schema create/update semantics

Schema reconciliation should be idempotent.

Expected behavior:

- if table does not exist and creation is allowed, create it once,
- if field does not exist and creation is allowed, create it once,
- if field exists and is already compatible, do nothing,
- if field exists and needs compatible update, modify it,
- if field exists but change is incompatible, follow `schema_conflict_policy`.

Schema changes should occur before record writes, never mid-record write.

---

14. Error Handling

14.1 Error categories

Configuration Errors

Examples:

- missing API key
- invalid table mapping
- unknown field type
- duplicate mapping definitions

Behavior:

- fail fast
- raise on startup or sync initialization

Validation Errors

Examples:

- invalid email
- bad enum value
- nil in non-nullable field
- serialization failure

Behavior:

- do not call Airtable API
- mark record failed
- persist field-level error details
- continue other records unless `fail_fast = true`

Schema Errors

Examples:

- failed to create table,
- failed to create field,
- failed to update field definition,
- incompatible existing field type,
- destructive change blocked by policy,
- remote metadata unavailable.

Behavior:

- classify as retryable or non-retryable,
- fail fast for incompatible schema conflicts,
- retry transient Airtable metadata/API failures,
- quarantine table mapping if integrity cannot be guaranteed.

Transport Errors

Examples:

- network timeout
- DNS issue
- connection reset

Behavior:

- retry with backoff
- mark retryable
- do not increment permanent failure count until retries exhausted

Remote API Errors

Examples:

- 401 unauthorized

- 403 forbidden
- 404 table not found
- 422 invalid request
- 429 rate limited
- 5xx Airtable server errors

Behavior:

- classify retryable vs non-retryable
- retry 429 and 5xx
- fail fast for auth/config issues

Consistency Errors

Examples:

- duplicate Airtable matches for one external ID
- Airtable returns unexpected response structure
- local mapping points to missing remote record

Behavior:

- quarantine record
- require manual intervention or repair task

14.2 Error object model

Provide structured errors:

- `RailsAirtableSync::ConfigurationError`
- `RailsAirtableSync::ValidationError`
- `RailsAirtableSync::SerializationError`
- `RailsAirtableSync::SchemaError`
- `RailsAirtableSync::SchemaConflictError`
- `RailsAirtableSync::SchemaMutationError`
- `RailsAirtableSync::TransportError`
- `RailsAirtableSync::ApiError`
- `RailsAirtableSync::RateLimitError`
- `RailsAirtableSync::ConsistencyError`

Field-level errors should include:

- model
- record id
- field name
- raw value
- expected type

- message

14.3 Retry policy

Default retryable conditions:

- timeouts
- connection failures
- 429
- 500, 502, 503, 504
- transient schema metadata/API failures

Retry strategy:

- exponential backoff with jitter
- capped retry count
- optional ActiveJob re-enqueue

Example:

- 1st retry: 1s
- 2nd retry: 2s
- 3rd retry: 4s plus jitter

14.4 Quarantine policy

A record or table mapping enters `quarantined` state when:

- duplicate external IDs found in Airtable,
- repeated permanent validation failures exceed threshold,
- mapping integrity cannot be guaranteed,
- schema conflict prevents safe sync and requires manual intervention.

Quarantined records are excluded from automatic retries until manually resolved.

---

15. Concurrency and Consistency

15.1 Concurrent sync protection

To avoid duplicate writes:

- lock local sync record during sync,
- or use advisory locks keyed by model + record ID.

Schema reconciliation should also use a table-level or mapping-level lock to avoid competing schema mutations.

15.2 Ordering guarantees

Per-record ordering should be preserved best-effort. If multiple jobs sync the same record concurrently, only one should proceed.

15.3 Transaction boundaries

Rails DB transaction should not wrap remote API call.

Recommended flow:

- read committed model state,
- perform API call,
- persist sync state in separate transaction.

Optionally, enqueue sync only after `after_commit`.

---

16. Background Jobs

16.1 Job integration

Support ActiveJob out of the box.

Example:

- `RailsAirtableSync::SyncJob`
- `RailsAirtableSync::BatchSyncJob`

16.2 Callback integration

Optional integration:

airtable_sync on: :commit

This should enqueue, not perform inline by default.

16.3 Backpressure

Batch jobs should respect Airtable rate limits:

- configurable batch size,
- configurable per-request sleep,
- auto-throttle after 429.

---

17. Logging, Metrics, and Instrumentation

17.1 Structured logs

Every sync attempt should log:

- model
- record id
- Airtable table
- operation (`create`, `update`, `skip`, `delete`)
- checksum_changed
- duration_ms
- result
- error class/message if any

Schema operations should also log:

- table name
- field name
- schema action (`inspect`, `create_table`, `create_field`, `update_field`, `block_change`)
- reconciliation outcome
- conflict policy applied

17.2 Notifications

Expose `ActiveSupport::Notifications` events:

- `airtable_sync.sync_started`
- `airtable_sync.sync_succeeded`
- `airtable_sync.sync_failed`
- `airtable_sync.record_skipped`
- `airtable_sync.schema_drift_detected`
- `airtable_sync.schema_inspection_started`
- `airtable_sync.schema_reconciled`
- `airtable_sync.schema_change_applied`

- `airtable_sync.schema_conflict_detected`
- `airtable_sync.schema_change_blocked`

17.3 Metrics

Recommended counters/histograms:

- `records_synced_total`
- `records_failed_total`
- `records_skipped_total`
- `api_requests_total`
- `api_retries_total`
- `sync_duration_ms`
- `validation_failures_total`
- `schema_tables_created_total`
- `schema_fields_created_total`
- `schema_fields_updated_total`
- `schema_conflicts_total`
- `schema_changes_blocked_total`

---

18. Security

18.1 Credentials

API keys must come from:

- Rails credentials
- environment variables
- injected secret store

Never log API keys.

18.2 PII handling

The gem should support:

- field redaction in logs
- configurable sensitive field list

18.3 Least privilege

Document recommendation to use an Airtable token scoped only to the required base/tables.

---

19. Configuration Options

```
RailsAirtableSync.configure do |config|
 config.api_key = ENV["AIRTABLE_API_KEY"]
 config.base_id = ENV["AIRTABLE_BASE_ID"]

 config.timeout = 10
 config.open_timeout = 3

 config.max_retries = 3
 config.retry_backoff = :exponential
 config.retry_jitter = true

 config.fail_fast = false
 config.enable_deletes = false
 config.delete_strategy = :archive_flag

 config.strict_types = true
 config.verify_after_write = true

 config.persist_sync_state = true
 config.use_advisory_locks = true

 config.auto_manage_schema = true
 config.auto_create_tables = true
 config.auto_create_fields = true
 config.auto_update_fields = true
 config.allow_destructive_schema_changes = false
 config.schema_conflict_policy = :fail

 config.on_validation_error = :mark_failed
 config.on_consistency_error = :quarantine

 config.logger = Rails.logger
end
```

---

20. Example Mapping

```
class Order < ApplicationRecord
```

```
 include RailsAirtableSync::Model

 airtable_sync table: "Orders" do
   scope { where.not(status: :draft) }

   record_key :id
   external_id_field "Order ID"

   field "Order ID", from: :id, type: :integer, nullable: false
   field "Customer Email", from: :customer_email, type: :email
   field "Status", from: :status, type: :single_select,
         allowed_values: %w[pending paid fulfilled cancelled]
   field "Total", from: :total_amount, type: :decimal
   field "Placed At", from: :placed_at, type: :datetime
   field "Line Items", from: :line_items_summary, type: :text

   checksum_fields :customer_email, :status, :total_amount, :placed_at,
:line_items_summary
 end
end
```

---

21. Testing Requirements

21.1 Unit tests

Must cover:

- type serialization
- nil handling
- enum validation
- checksum generation
- retry classification
- mapping validation
- schema type mapping
- schema reconciliation decision logic

21.2 Integration tests

Must cover:

- create flow
- update flow
- unchanged skip flow
- retry after transient failure

- quarantine on duplicate match
- remote schema drift
- table auto-creation
- field auto-creation
- compatible field auto-update
- blocked destructive schema change

21.3 Failure-mode tests

Must cover:

- invalid credentials
- missing table
- rate limiting
- network timeout
- malformed API response
- concurrent job contention
- schema metadata fetch failure
- incompatible field type conflict

21.4 Contract tests

Optional but recommended against Airtable API sandbox/base.

---

22. Acceptance Criteria

The gem is acceptable if it can:

1. Sync a configured Rails model to Airtable without duplicates.
2. Correctly serialize common Rails field types to Airtable-compatible values.
3. Reject invalid payloads before API submission in strict mode.
4. Skip unchanged records using deterministic checksums.
5. Recover from transient failures with retries.
6. Persist sync state and error details locally.
7. Quarantine records when integrity cannot be guaranteed.
8. Preserve one-way source-of-truth semantics from Rails to Airtable.
9. Expose meaningful logs and instrumentation.
10. Avoid destructive behavior unless explicitly configured.
11. Auto-create missing Airtable tables and fields from Rails mappings by default.
12. Apply non-destructive schema updates automatically.
13. Block unsafe destructive schema changes unless explicitly enabled.
14. Prevent record sync from proceeding when schema integrity cannot be guaranteed.

---

23. Recommended Future Enhancements

- schema introspection assistant
- admin dashboard for failed/quarantined records
- bulk sync optimization
- selective field-level overwrite policies
- support for attachments upload workflows
- diff reports before sync
- circuit breaker for repeated Airtable outages

---

24. Summary Design Recommendation

For reliability, the best implementation shape is:

- **explicit DSL mappings**
- **local sync state table**
- **strict serializer layer**
- **default schema auto-reconciliation**
- **guardrails for destructive mutations**
- **checksum-based idempotency**
- **advisory locks**
- **retryable job execution**
- **quarantine for integrity threats**
- **structured logging and notifications**